

The time has come to add another language to our programming “tool belt.” This is a skill that you are going to have to get used to because learning a new programming language is something that a programmer does pretty frequently. Once you have a good understanding of a general purpose programming language (e.g., Python), transitioning to another language is really just a matter of identifying the subtle differences in syntax. It therefore takes a considerably shorter time than learning your first programming language.

Why Java?

Before we answer this question, let's discuss why we need to learn any new programming language in the first place. If Python is as powerful and robust as we've been claiming, why can't we use it for everything? As a programmer, you will spend a lot of your time catering to the needs of a client, and those needs are rarely the same. Sometimes, your tasks will involve maintaining code that was written decades ago in a programming language that is now obsolete. Other times, it will involve working on a small part of a larger system, and the client will require that you work in a language that is consistent with the rest of the system. In these cases, it is important that you not only have some form of experience in multiple languages, but also be able to quickly pick up and use new (to you) programming languages. Additionally, newer languages are always popping up, and you may need to learn them in order to keep relevant in this field.

So back to the original question: Why Java? First, it is considered one of the most widely used programming languages in industry. Second, Java is platform independent. Many programming languages generate compiled executables that are suitable only for the target system on which the code was compiled. This means that the executable code will look different depending on the target system. Java works a bit differently. Java source code is compiled to a platform independent intermediate language known as **bytecodes**. The bytecodes may then be distributed to any target system. To execute the bytecodes, the target system must have a Java virtual machine. One doesn't need access to the original source code to execute a Java program. Instead, only the bytecodes are necessary. Third, Java is secure. In addition to allowing for the distribution of bytecodes without the source code, it protects novice programmers from dealing with many details “under the hood,” and this provides an extra level of security.

How is Java installed?

The instructions for installing Java will vary from system to system as it depends on the operating system (e.g., Windows, Linux, MacOS, etc) and the CPU of the target computer (i.e., 32-bit or 64-bit). There are many online tutorials that show how to install Java on a variety of systems. Google it! Note, however, that you will want the Java Development Kit (JDK). It comes with the tools necessary to compile Java source code to bytecodes. In addition, it comes with the Java Virtual Machine (JVM) that can execute the bytecodes. Do not confuse the JDK with the JRE (Java runtime Environment) that only contains the JVM (i.e., no tools to compile Java source code).

The JDK can be downloaded at: <https://www.oracle.com/java/technologies/javase-downloads.html>

A first Java program

When you were introduced to Python, the concept of an IDE (Integrated Development Environment) was also covered. An IDE is a form of editor that allows you to write programs in one or more programming languages. It normally has features such as syntax highlighting, syntax suggestions, and allows you to compile and/or execute the programs that you have written easily. For Python, the default IDE on the RPi is IDLE. Unfortunately, IDLE only works with Python.

There are IDEs that are more general purpose than IDLE, in that they support more than one programming language. Examples include Eclipse, NetBeans, and Code::Blocks. The process of installing and setting up these IDEs is beyond the scope of this lesson. If you want to work on large projects in Java and other general purpose programming languages, you will want to research how to install a robust IDE on your system. As an FYI, Eclipse is perhaps the most widely used IDE for Java. For the purpose of this lesson, it is significantly easier to use a general purpose text editor, and compile Java source code and execute the resulting Java bytecodes at the command line (i.e., in the terminal or console).

One of the main features of Java is that it is fully object-oriented. This means that everything that you do within Java has to be defined in a class. It embodies the principle that everything is an object! Even a simple hello world program must be in a class. Unlike Python, where indentation is used to identify blocks of statements, Java used curly braces. The left (or open) curly brace is typically placed after the header of a block and shows where the block begins, and the right (or close) curly brace shows where the block ends. Here's an example:

```
class HelloWorld
{
    ...
}
```

Note that there is still indentation within the block. Although this is not required, it is considered good programming practice to do so. In fact, not doing so is indicative of a worthless programmer. Well, maybe that's too harsh. Maybe not...

You'll notice the keyword `class`, which signifies that the word coming after it is the name of a class. This is similar to Python. The name of the class can be any identifier. The only restrictions on identifiers is that they have to be made up of alphanumeric characters and underscores, and cannot begin with a digit. It is Java convention to initially capitalize class names; however, that's not enforced. Since it is convention, however, it does make your code easier to read by professional programmers.

Unlike Python, where statements don't necessarily have to be encapsulated in methods, Java requires everything to be in at least one method. Method signatures have the following form:

```
modifier return_type method_name(parameter_list)
{
    method body
}
```

The **modifier** is a word that determines the accessibility (or usability) of a method. The most common modifiers are:

- `public`: indicates that the method can be accessed from anywhere; and
- `private`: indicates that the method can only be accessed within its defining class.

The **return_type** is a key word that informs the compiler of the kind of result that the function returns to whichever statement called the function in the first place. In Python, there is no such thing as formally defined data types. A Python variable can take on any type, and a Python function can return any type (even multiple types). In Java, variables must be declared along with their type. These are valid throughout the duration of the program and cannot be changed! Functions may return a value, and if so, the return type must be specified. If the function does not return anything, then the keyword `void` is used. Common return types are `int`, `long`, `float`, `double`, `String`, and `char`. We will cover some of these later in this lesson. The return type can also be a class. That is, the value returned can be an instance of a class!

The **method_name** can be any valid Java identifier. It is convention that it begin with a lower case character, and that successive words be initially capitalized (e.g., `printGrades`). This way, it is easy to differentiate between a class name and a function name.

The **parameter_list** is a comma separated list of variables and their types that are passed in as inputs to the function. If a function does not require any parameters, then the parentheses are left empty. Below are some examples of method signatures:

- `public int getSum(int a, int b)`
- `public String printBoard()`
- `private double getAverage(double a, double b, double c)`
- `private void doSomethingCool(int a, double c, String b, char d)`

Since a program's source code in Java must be specified within at least one function, how does the compiler know which function is to be executed first? There is a special function with a special name that is always executed first in a Java program. That function has a specific signature:

```
public static void main (String[] args)
```

Note that the method is `public`, and can therefore be accessed outside of the class. It does not return anything due to the keyword `void`. Its name is `main`, and it takes an array of `Strings` called `args`. The square brackets after the `String` type indicates that it is an array of `Strings`. Arrays in Java are similar to lists in Python. The `static` keyword enforces the idea that, even if multiple objects were created from the class in which the `main` function is located, there will only be one copy of `main` that is the same for all the objects. That is, the `main` function is shared among all instances of the class that contains it. The `main` function is considered the entry point of a program.

Let's put all of this information together to create our first "hello world" Java program:

```
public class HelloWorld
{
    public static void main (String[] args)
    {
        System.out.println("Programming rules, man!");
    }
}
```

Note the semicolon at the end of the statement that displays “Programming rules, man!” In Java (and, in fact, many other programming languages), the end of a statement is marked by a semicolon.

Did you know?

A missing semicolon at the end of a Java statement is the most common syntax error. It is practically guaranteed that you will forget to put a semicolon at the end of a Java statement at some point.

Let's discuss the content of the source code to the simple hello world program. There is really only a single statement in the program: the one that displays the string. It is enclosed in the main function, which itself is enclosed in the HelloWorld class. Let's formally discuss the single statement:

```
System.out.println("Programming rules, man!");
```

In order to access the members of classes, we use the dot notation. Recall that we used the dot notation to access members within classes in Python as well. In Java, the `println` function displays the parameter specified to it to standard output (i.e., to the console). To access it, we fully qualify it with `System.out.println`. Technically, `System` is a class, inside of which is an object called `out`. The defining class of the `out` object provides the `println` function.

Note that the `println` function automatically inserts a newline after displaying its parameter to the console. This is similar to the behavior of the `print` statement in Python. To suppress the newline, use the function `print` instead of `println`. For example:

```
System.out.print("Programming rules, man!");
```

So how do we compile and execute our first Java program? Here is this process from the command line:

```
$ javac HelloWorld.java
$ java HelloWorld
Programming rules, man!
```

On some operating systems (e.g., Windows) it may be necessary to include the location of the Java compiler and other executables in the *path* environment variable in order to allow compilation and execution from any folder. To compile a Java program from the command line, we use the following syntax:

```
javac myfile.java
```

Note that it is convention to name our Java source files with a `.java` extension. The compilation process generates a new file of the same name; however, it has a `.class` extension. To execute the compiled Java bytecodes, we use the following syntax:

```
java myfile
```

Note that we do not specify the `.class` extension when executing the bytecodes!

Here's an example of what happens when you try to compile a Java program with a syntax error. First, here's a modified version of the hello world program:

```

public class HelloWorld
{
    public static void main (String[] args)
    {
        System.out.println("Programming rules, man!")
    }
}

```

Did you notice the error? Here's the output generated when compiling this modified program:

```

$ javac HelloWorld.java
HelloWorld.java:5: error: ';' expected
    System.out.println("Programming rules, man!")
                                ^
1 error

```

The error is actually quite easy to see when the program is compiled. The Java compiler does a pretty good job of informing us what and where the error is: a missing semicolon at the end of the statement that displays output to the console.

Data types, constants, and variables

Although Python does not support the formal declaration of variables with data types, the data types in Java are actually very similar to those in Python. Java supports primitive types such as integers (`int`), long integers (`long`), single precision floating point numbers (`float`), double precision floating point numbers (`double`), characters (`char`), Booleans (`boolean`), and more. In addition it supports more complex types such as a `String` (which, as in Python, is a *string* of characters) and user-defined types defined in classes that can be instantiated as objects.

When it comes to the variables and the way that they are used, there is one difference with Java. In fact, this difference was something that we dealt with in Scratch. Before a variable can be used, it has to be declared. This means that the compiler needs to know what data type and name will be associated with a variable. Once it has been declared as one type, it can not be reused to store a different data type.

Declaring a variable can be done with a statement in the following format:

```
data_type variable_name;
```

The statements below are examples of valid variable declarations:

- `int x;`
- `float num1, num2;`
- `double c;`
- `String str;`
- `char c;`
- `boolean b;`

Note the second example. Here, two variables are actually declared, `num1` and `num2`, both of the data type `float`. Once declared, the variables can be assigned and used as they would be in Python (i.e.,

they do not have to be redeclared every time we wish to use one). In fact, we can assign new values to previously declared (and even previously initialized) variables at any point in a program. It is also possible to declare and initialize a variable with a value in a single Java statement, as shown in the examples below:

- `int x = 3;`
- `float num1 = 3.14, num2 = 1.414;`
- `double c = 2.78;`
- `String str = "hard drive";`
- `char c = 'p';`
- `boolean b = false;`

It is also possible to assign the result of an expression to a variable, even as you declare it:

- `double d = 1.3 + 2.5;`
- `char grade = calcGrade();`

Note that the function `calcGrade` in the second example must return a value of type `char`.

Did you know?

While the character `#` is used to mark single-line comments in Python, Java uses `//`. For example:

```
// this is a comment
```

Java also uses `/*` to begin a multi-line comment and `*/` to end a multi-line comment. For example:

```
/* this is
   a multi-line
   comment */
```

Of course it is possible to use `//` for a comment spanning multiple lines so long as each line begins with `//`.

Now that we have a general understanding of primitive data types and how to use them, let's write a slightly more complicated program that displays the sum of two integers to the terminal.

```
public class Sum
{
    public static void main(String[] args)
    {
        int a = 5;
        int b = 7;

        System.out.println(a + " + " + b + " = " + (a + b));
    }
}
```

Here is the output of this program's compilation and execution:

```
$ javac Sum.java
$ java Sum
5 + 7 = 12
```

Let's look more closely at the following statement:

```
System.out.println(a + " + " + b + " = " + (a + b));
```

The variable *a* is an integer. It appears to be added to the string “+”. However, Java cannot add an integer to a string; therefore, it converts the integer to a string first and then concatenates the two strings. Since the value of *a* is 5, then *a* + “+” results in “5” + “+” which yields the string “5 +”. In this statement, the integers are all converted to strings first before concatenating them together.

Constants in Java are declared using the “final” keyword. Here is an example:

```
public static final int NUM_RECORDS = 100;
```

It is convention to capitalize the entire constant name and use an underscore to separate words. The constant must be assigned a value, but exactly where you are allowed to give it a value depends on where and how you define it (e.g. a class local variable defined as a constant will need to be given a value on the same line as it is defined but an instance variable defined as a constant can be given a value in the constructor).

Activity 1: What's my grade?

Let's write a simple program that calculates the letter grade corresponding to a student's score on a test. The program will contain two functions. The bulk of the code will be written in the main function; however, it will take advantage of a `calcGrade` function that takes the score (a double) as its parameter and returns the matching letter grade. Here is the source code:

```
public class LetterGrade
{
    public static void main(String[] args)
    {
        double score = 87.5;
        char letter_grade;

        letter_grade = calcGrade(score);

        System.out.println("Your grade is " + letter_grade);
    }

    public static char calcGrade(double d)
    {
        if (d >= 90.0)
            return 'A';
        else if (d >= 80.0)
            return 'B';
        else if (d >= 70.0)
            return 'C';
        else if (d >= 60.0)
            return 'D';
        else
            return 'F';
    }
}
```

```
    }  
}
```

And here is the output:

```
$ java LetterGrade  
Your grade is B
```

Although the syntax is different, you should be able to use your existing knowledge of Python in order to understand what is going on. In Java, we use `else if` instead of `elif` to add a condition to the `else` clause of an `if` statement. In fact, we can chain them (just as we do with `elif` in Python). Note that Java does not require a colon at the end of an `if` statement.

If the true or false (else) part of an `if` statement only includes a single statement, then curly braces can be omitted. By default, a block contains a single statement. If, however, multiple statements were required, curly braces would be needed. Here's an example of a multi-line `if else` statement:

```
if (d >= 90)  
{  
    System.out.println("Wow, good job!");  
    return 'A';  
}
```

Compare this with a similar block of statements in Python:

```
if (d >= 90):  
    print("Wow, good job!")  
    return 'A'
```

Input and output

You have already seen how output can be done in Java using `System.out.print` and `System.out.println`. This is the standard way to direct output to standard output. To obtain input from standard input, we can use `System.in` (the inverse of `System.out`).

Although it is not as straightforward, obtaining user input can be accomplished as follows:

```
import java.util.Scanner;  
  
public class Input  
{  
    public static void main(String[] args)  
    {  
        Scanner s = new Scanner(System.in);  
  
        while (s.hasNextLine())  
        {  
            String line = s.nextLine();  
  
            System.out.println(line);  
        }  
    }  
}
```



```
}
```

First, note that the `import` keyword is also used in Java to bring in external libraries. In fact, it's not much different than in Python! The statement `import java.util.Scanner` imports the `Scanner` class from the `util` library in Java. To setup user input, we instantiate a new `Scanner`, using `System.in` as the parameter (i.e., we can now scan for user input from standard input). Note the use of the keyword `new`. In Java, the keyword `new` is used to create a new instance of a class. The `Scanner` class has several useful methods. The first that we will make use of is `hasNextLine`, which returns `true` if the scanner detects that another line of input exists. The second is `nextLine`, which returns the next line of input. This simple program obtains user input and echoes it back to the console.

Operators

For the most part, the operators that you learned in Python are the same (or very similar) in Java as well. This means that it should be relatively straightforward to convert code that you've written in Python to Java. There are, however, a few exceptions that we will now discuss.

The typical **arithmetic operators** for addition (+), subtraction (-), multiplication (*), division (/), and modulus (%) are exactly the same in Java. Java, however, does not use `**` as the exponentiation operator. Instead, it makes use of a math library. Specifically, a function called `pow` can be called as follows to replicate the expression `2 ** 3` in Python:

```
Math.pow(2, 3);
```

Another difference in the arithmetic operators is the floor division. Python has a floor division operator (`//`). As you have seen, this is used in Java to denote a single line comment. To obtain floor division in Java, we need to make use of the math library again, as follows:

```
Math.floor(5.2 / 8.11);
```

Note that, if the values that you are dividing are both integers, (i.e., whole numbers), then the result of their division will be an integer! This is the same result as a floor division. In fact, this is something that you should be very careful about in Java, because it is often the source of logical errors that are very difficult to trace. Java's division operator will give different results for integers than it would for floating point numbers. Take a look at the examples below:

- $1 / 2 \rightarrow 0$
- $1.0 / 2 \rightarrow 0.5$
- $1 / 2.0 \rightarrow 0.5$
- $1.0 / 2.0 \rightarrow 0.5$
- $(\text{float})1 / 2 \rightarrow 0.5$
- $1 / (\text{float})2 \rightarrow 0.5$

The first example is a division of integers which will always return an integer result that is equal to the quotient of the division. If the remainder is needed, we can use the modulus operator. The second, third, and fourth examples are divisions which involve a floating point value as one (or both) of the operands. In this case, the division operator returns the expected floating point result. The fifth and sixth examples result in floating point division because one of the operators is converted or **typecast** to a floating point value. Typecasting is used to inform the compiler that the variable that it applies to is to

be considered of the type specified in the parentheses. Depending on the direction of the typecast, typecasting can result in a loss or increase in accuracy of the operation.

The following table enumerates all of Java's arithmetic operators (assume that $a = 23$, $b = 17$, $c = 4.0$, and $d = 8.0$):

Java Arithmetic Operators and Examples			
+	addition	$a + b = 40$	$c + d = 12.0$
-	subtraction	$a - b = 6$	$c - d = -4.0$
*	multiplication	$a * b = 391$	$c * d = 32.0$
/	division	$a / b = 1$	$c / d = 0.5$
%	modulus	$a \% b = 6$	$c \% d = 4.0$

All but one of the Python **relational operators** apply in Java. Recall that Python provides two relational operators to check for inequality: `!=` and `<>`. Java does not support the latter (i.e., it only supports `!=`). The following table enumerates all of Java's relational operators (assume that $a = 23$ and $b = 17$):

Java Relational Operators and Examples		
<code>==</code>	equality	$a == b$ is false
<code>!=</code>	inequality	$a != b$ is true
<code>></code>	greater than	$a > b$ is true
<code><</code>	less than	$a < b$ is false
<code>>=</code>	greater than or equal to	$a >= b$ is true
<code><=</code>	less than or equal to	$a <= b$ is false

Java has special **prefix and postfix operators** that you most likely have not seen before (certainly not in Python, anyways). These operators are often referred to as syntactic sugar, in that they provide a shorthand for incrementing or decrementing a variable. Here is a snippet of code as an example:

```
int p = 3;
p++; // p is now 4
++p; // p is now 5
int q = 5;
q--; // q is now 4
--q; // q is now 3
```

The `++` and `--` operators are used to increment and decrement a variable respectively. They only increment or decrement by 1 in each case. The location of the `++` or `--` operator relative to the variable determines the order in which this increment or decrement is done and is dependent on the context in which it is used. If the operator is placed before the variable (i.e., as a prefix), then it means that the increment or decrement is carried out before the variable is used. Conversely, if the operator is placed after the variable (i.e., postfix), then it means that the increment or decrement is carried out after the variable is used. Here's an example of the nuances between prefix and postfix:

```

int p = 3;
System.out.println(p++);    // p is displayed and then
                             // incremented to 4

System.out.println(p);
System.out.println(++p);    // p is incremented to 5 and
                             // then displayed

```

And here's the output:

```

$ java Arithmetic
3
4
5

```

Note that the following statements all accomplish the same thing: increment the value of the variable *p* by 1:

- `p = p + 1;`
- `p += 1;`
- `p++;`
- `++p;`

The first two are also valid in Python (and do the same thing), although no semicolons are required.

Java **assignment operators** are similar to those in Python, except that there is no assignment operator for exponentiation or floor division. The following table enumerates all of Java's assignment operators (assume that *a* = 23.0 and *b* = 17):

Java Assignment Operators and Examples		
=	<i>a</i> = <i>b</i> assigns <i>b</i> to <i>a</i>	<i>a</i> = 17.0
+=	<i>a</i> += <i>b</i> increments <i>a</i> by <i>b</i> (same as <i>a</i> = <i>a</i> + <i>b</i>)	<i>a</i> = 40.0
-=	<i>a</i> -= <i>b</i> decrements <i>a</i> by <i>b</i> (same as <i>a</i> = <i>a</i> - <i>b</i>)	<i>a</i> = 6.0
*=	<i>a</i> *= <i>b</i> multiplies <i>a</i> by <i>b</i> and stores the result in <i>a</i> (same as <i>a</i> = <i>a</i> * <i>b</i>)	<i>a</i> = 391.0
/=	<i>a</i> /= <i>b</i> divides <i>a</i> by <i>b</i> and stores the result in <i>a</i> (same as <i>a</i> = <i>a</i> / <i>b</i>)	<i>a</i> = 1.3529411764705883
%=	<i>a</i> %= <i>b</i> divides <i>a</i> by <i>b</i> and stores the remainder in <i>a</i> (same as <i>a</i> = <i>a</i> % <i>b</i>)	<i>a</i> = 6.0

In Java, **bitwise operators** work exactly as they do in Python, on bits and perform bit-by-bit operations. The following table enumerates all of Java's assignment operators (assume that *a* = 60, or 00111100 in binary, and *b* = 13, or 00001101 in binary):

Java Bitwise Operators and Examples		
&	bitwise <i>and</i>	<i>a</i> & <i>b</i> = 00001100 (or 12 in decimal)
	bitwise <i>or</i>	<i>a</i> <i>b</i> = 00111101 (or 61 in decimal)
^	bitwise <i>xor</i>	<i>a</i> ^ <i>b</i> = 00110001 (or 49 in decimal)

~	bitwise <i>not</i>	~a = 11000011 (or -61 in decimal; we will explain this one later)
<<	left shift	a << 2 = 11110000 (or 240 in decimal)
>>	right shift	a >> 2 = 1111 (or 15 in decimal)

In Python, we had words to represent the **logical operators** *and*, *or*, and *not*. In Java, we have symbols to represent these same operators. Logical operators operate on conditions and provide the overall logical result. The following table enumerates all of Java's logical operators (assume that a is true and b is false):

Java Logical Operators and Examples		
&&	logical <i>and</i>	a && b is false
	logical <i>or</i>	a b is true
!	logical <i>not</i>	!a is false; !b is true

The logical operators are most often used with expressions in the conditions of if statements and loop constructs.

Primary control constructs

Similar to Python, Java uses **sequence** as a primary control construct. Statements are executed, one after another. Control flow is as expected. It can, however, be changed when, for example, functions are called.

Selection in Java is very similar to Python and is implemented as an if statement. As shown earlier, if statements are nearly the same; however, no colon is used at the end of the if statement, braces are used to denote blocks, and `else if` is used instead of `elif`. Note that the conditions of if statements must be enclosed in parentheses:

```

if (grade > 89.5)
    letter_grade = 'A'
else if (grade > 79.5)
    letter_grade = 'B'
else if (grade > 69.5)
    letter_grade = 'C'
else if (grade > 59.5)
    letter_grade = 'D'
else
    letter_grade = 'F'

```

In Python, strings can be enclosed in single or double quotes. In Java, however, they must always be enclosed in double quotes; for example:

```
"This is a string."
```

In Java, characters are always enclosed in single quotes; for example:

```
'^'
```

There is another selection construct in Java called a switch-case. Let's explain it by starting with an example:

```
char grade = 'B';

switch (grade)
{
    case 'A':
        System.out.println("Excellent job!");
        break;
    case 'B':
        System.out.println("Great job!");
        break;
    case 'C':
        System.out.println("Good job!");
        break;
    case 'D':
        System.out.println("Fair job!");
        break;
    case 'F':
        System.out.println("Poor job!");
        break;
    default:
        System.out.println("Invalid grade!");
        break;
}
```

A switch statement is very much like an if-else with many possible else-if clauses. The condition is restricted to variables of the type `int`, `char`, and `String` (among a few others that are not used often). In the example above, the variable that serves as the condition value is *grade*. If the grade is A, then the first case is *triggered* (and the string “Excellent job!” is echoed to the console). The `break` statement then exits the switch construct so that none of the other cases are executed. In a switch statement, cases can *fall through* each other. Here's an example, where earning an A or B yields the string “Great job!”, a C yields the string “Good job!”, and a D or F yields the string “Poor job!”:

```
char grade = 'B';

switch (grade)
{
    case 'A':
    case 'B':
        System.out.println("Great job!");
        break;
    case 'C':
        System.out.println("Good job!");
        break;
    case 'D':
    case 'F':
        System.out.println("Poor job!");
}
```

```

        break;
    default:
        System.out.println("Invalid grade!");
        break;
}

```

The default case is a *catch-all* that executes if none of the other cases are evaluated to be true. If, for example, the grade provided was mistakenly entered as 'E', then the output would be “Invalid grade!”

Repetition in Java is somewhat similar to Python. There are three repetition constructs. The while-do loop is similar to Python. It is formatted as follows:

```

while (condition)
{
    loop_body
}

```

Note that the braces are only required if there is more than one statement in the loop body. Here's an example of a single statement in the loop body that does not require braces:

```

int sum = 0;
while (sum > 0)
    sum--;

```

In the example above, the loop body is never executed because the variable *sum* is initialized to 0. There is a variation of the while-do loop called the do-while loop that is formatted as follows:

```

do
{
    loop_body
} while (condition);

```

The main differences of this form of the while loop are that (1) the loop body always executes at least one time; and (2) there is a semicolon after the condition at the bottom of the loop. Here's an example:

```

int sum = 0;
do
{
    sum--;
} while (sum > 0);

```

In this example, the loop body executes once. When it does, the value of the variable *sum* is decremented by one (to -1), and the condition at the bottom of the loop becomes false (which breaks out of the while loop).

In Java, the for loop is drastically different than it is in Python. It is formatted as follows:

```

for (initialization; condition; change)
{
    loop_body
}

```

The initialization portion is used to initialize one or more variables (usually used as a counter). The condition portion is evaluated and decides if the loop body is to be executed (if the condition evaluates to true). The change portion changes the value of one or more of the variables in the initialization portion (e.g., increment or decrement). This occurs after each execution of the loop body. Here's an example:

```
int i;

for (i=0; i<10; i++)
{
    System.out.print((i * i) + " ");
}
```

This example changes the value of the variable *i* from 0 to 10. The loop body is executed when *i* is between 0 and 9 inclusive. When its value is 10, the loop body is not executed. Initially, the value of *i* is initialized to 0. Since 0 is less than 10, the loop body is executed. After this, *i* is incremented (to 1). Since 1 is less than 10, the loop body is executed again. This continues. At one point, *i* has the value 9. Since 9 is less than 10, the loop body is executed. After this, *i* is incremented (to 10). Since 10 is not less than 10, the for loop is then terminated, and control continues to the statement that follows it.

Formally, the initialization is done before the first iteration. The condition is evaluated before each iteration and determines whether the loop body is executed. The change is carried out at the end of each iteration. While the majority of your for loops will have initialization, condition, and change dealing with the same variable, this is not a rule. The change portion can vary greatly. Sometimes, it represents an increment (as in the example above). At other times, it could be a decrement or even an entirely different arithmetic expression. The condition portion, however, should always evaluate either to true or false. The for loop is exited when the condition evaluates to false.

Additionally, it is possible to leave some or all of the parts of the for loop (i.e., initialization, condition, and change) empty. There must always be semicolons that delineate the three portions of the for loop, however, even if some are empty. If one selects to have one or more of the portions empty, a way to cater for the missing portion(s) must be specified. Here's an example:

```
int i = 10;

for (;;)
{
    i--;
    if (i == 0)
        break;
}
```

The for loop header, `for (;;)`, is technically an infinite loop. Therefore, there must be some way to exit the loop in the loop body. This is accomplished as the variable *i* is decremented at each iteration of the loop. It will eventually reach the value 0; and when that occurs, the condition of the if statement will evaluate to true, and the `break` keyword will terminate the loop.

Lastly, it is also possible to place more than one initialization in the initialization portion, more than one condition in the condition portion, and more than one change in the change portion. The only requirement is that the different components are separated by a comma. Also, variables can actually be declared in the initialization portion, eliminating the need to do so beforehand. Here is an example of both of these:

```
System.out.println("Before loop");
for (int i=0, j=10; i<=100 && j >= 0; i+=2, j-=2)
{
    System.out.println("In loop; i=" + i + ", j=" + j);
}
System.out.println("After loop");
```

Note that, if variables are declared in the initialization portion of the for loop, then they become local to the loop and are unavailable after the loop. Let's explain the snippet of code above. First, the variables *i* and *j* are declared and initialized to 0 and 10 respectively in the initialization portion of the for loop. Second, if *i* is less than or equal to 100 and *j* is greater than or equal to 0, then the loop body is executed. Third, after the loop body is executed, *i* is incremented by 2 and *j* is decremented by 2.

Here is the output:

```
$ java Loop
Before loop
In loop; i=0, j=10
In loop; i=2, j=8
In loop; i=4, j=6
In loop; i=6, j=4
In loop; i=8, j=2
In loop; i=10, j=0
After loop
```

There is another variation of the for loop, called the foreach loop, that will be discussed later in this lesson.

Variable scope

In Java, scoping of variables is similar to Python. That is, global variables can be declared. They must be declared in the class, but outside of any function in the class. This makes them class variables (which you already know about). Technically, they are global to the class. Variables declared in functions are local to those functions.

Program flow

Let's use a previous example to illustrate program flow in Java (which, by the way, is similar to Python):


```

public class LetterGrade
{
    public static void main(String[] args)
    {
1:         double score = 87.5;
2:         char letter_grade;
3:         letter_grade = calcGrade(score);

4:         System.out.println("Your grade is " + letter_grade);
    }

    public static char calcGrade(double d)
    {
5:         if (d >= 90.0)
6:             return 'A';
7:         else if (d >= 80.0)
8:             return 'B';
9:         else if (d >= 70.0)
10:            return 'C';
11:        else if (d >= 60.0)
12:            return 'D';
13:        else
14:            return 'F';
    }
}

```

Using the source code above, in the space below try to number the statements in the order that they would be executed:

Note that the main function doesn't necessarily have to be placed as the first function in a class. Functions can be placed in any order! Here's a slightly modified version of the program above:

```

public class LetterGrade
{
    public static char calcGrade(double d)
    {
1:         if (d >= 90.0)
2:             return 'A';
3:         else if (d >= 80.0)
4:             return 'B';
5:         else if (d >= 70.0)
6:             return 'C';
7:         else if (d >= 60.0)
8:             return 'D';
9:         else

```

```

10:         return 'F';
    }

    public static void main(String[] args)
    {
11:         double score = 66.0;
12:         char letter_grade;

13:         letter_grade = calcGrade(score);

14:         System.out.println("Your grade is " + letter_grade);
    }
}

```

Using the modified source code above, in the space below try to number the statements in the order that they would be executed:

Arrays

By now, you are familiar with Python lists. In previous lessons, we mentioned that a list in Python is similar to an array. The main differences are that (1) a Python list can store heterogeneous pieces of data, while arrays are designed to store homogeneous pieces of data (i.e., values of the same data type); and (2) arrays must be declared with a fixed capacity (i.e., they cannot grow as needed).

Java does not support lists in the same way that Python does. Instead, it support arrays as described above. In the end, they are almost the same. Declaring an array in Java is very similar to declaring any other data type. The difference is that square brackets are placed between the data type and the identifier (i.e., the variable name). Here is an example that declares two variables: (1) an integer, x; and (2) an array, y:

```

int x;
int[] y;

```

Just as with simple variables, arrays can be declared and initialized in the same statement:

```

int x = 10;
int[] y = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

```

The second statement in the example above declares an array of integers and initializes it with the integers 1 through 10 at indexes 0 through 9 respectively. This sets the array's capacity at 10 values. The curly braces allow for the initialization of arrays upon declaration.

Once arrays have been declared, its individual elements can be accessed in a manner similar to accessing the elements of a list in Python (i.e., using the index in square brackets following the name of the array).

Here's an example:

```
System.out.println(y[3]);    // outputs 4
y[6] = -1;                  // array is now { 1, 2, 3, 4, 5, 6, -1, 8, 9, 10 }
```

Note that array slices are not supported in Java. This is rather unfortunate! It is also possible to just declare an array and subsequently initialize it as follows:

```
int[] y = new int[10];

for (int i = 0; i < 10; i++)
    y[i] = i * i;
```

This snippet of code declares an array of 10 integers. It then iteratively populates the array with the first ten squares (i.e., $0 * 0$, $1 * 1$, $2 * 2$, etc).

Here are a few other differences between Python lists and Java arrays:

- There is no way to actually delete an element from an array (instead, we can zero out an element or shift elements over to effectively delete an element); and
- Similarly, there is no way to insert an element in an array (once the capacity is set, it cannot be changed).

Can you understand what the source code below produces and how it works?

```
public class Arrays
{
    public static void main(String[] args)
    {
        int[] test1 = { 67, 23, 98, 45, 99, 45 };
        int[] test2 = { 88, 44, 94, 78, 64, 44 };
        int i = 0;

        while (i < 6)
        {
            if (test1[i] > 80 && test2[i] > 80)
            {
                System.out.println("Student " + i +
                                   " doesn't need to take the final");
            }
            else if (test1[i] > 80 || test2[i] > 80)
            {
                System.out.println("Student " + i +
                                   " can decide to take the final");
            }
            else if (!(test1[i] > 60) && !(test2[i] > 60))
            {
                System.out.println("Student " + i +
                                   " definitely needs to take the final");
            }
        }
    }
}
```

```

    }
    else
    {
        System.out.println("Student " + i +
            " -- no decision");
    }

    i++;
}
}
}

```

You will notice that statements in Java can be split among multiple lines without needing any special formatting or continuation character. This is evident in the statements outputting strings to the console via `System.out.println`. In Python, a backslash is required to inform the interpreter that a statement is continued on the next line.

The program first declares and initializes two arrays of six integers. The first, `test1`, represents six students' grades for the first test. The second, `test2`, represents the same six students' grades for the second test. The while loop then iterates over all of the students' grades (there are two of them for all six students). Next, an informative message is displayed depending on the test grades of each student. If a student earned more than an 80 on both tests, then the student doesn't need to take the final. If a student earned more than an 80 on one of the tests but not the other, then the student can decide to take the final. If a student did not earn more than a 60 on both tests (i.e., earned 60 or less on both tests), then the student must take the final. If none of these cases is true, then there is no decision for the student.

The first student, represented by the values in the arrays at the first index (i.e., index 0), earned a 67 on the first test and an 88 on the second test. Following the logic, this student “can decide to take the final.” The second student, represented by the values in the arrays at the second index, earned a 23 on the first test and a 44 on the second test. Following the logic, this student “definitely needs to take the final.” The third student earned a 98 and a 94. This student “doesn't need to take the final.” The fourth student earned a 45 and a 78, which results in “no decision.” The fifth student earned a 99 and a 64, and “can decide to take the final.” The sixth student earned a 45 and a 44, and “definitely needs to take the final.”

To confirm this logic, here is the output:

```

$ java Arrays
Student 0 can decide to take the final
Student 1 definitely needs to take the final
Student 2 doesn't need to take the final
Student 3 -- no decision
Student 4 can decide to take the final
Student 5 definitely needs to take the final

```

The foreach loop

Earlier, the foreach loop was briefly mentioned as another variation of the for loop. Formally, it allows iteration over the items in any type of collection (e.g., an array). Java has many collections, and

discussing them is beyond the scope of this lesson. However, let's take a look at how the foreach loop can be used to iterate over the items in an array. Here's an example:

```
double[] cool_constants = new double[5];

cool_constants[0] = 3.14159;    // pi
cool_constants[1] = 2.71828;    // e
cool_constants[2] = 1.414;      // sqrt(2)
cool_constants[3] = 1.73;       // sqrt(3)
cool_constants[4] = 1.618;      // phi

for (double d : cool_constants)
    System.out.println(d);
```

The foreach loop actually uses the keyword `for` (and not literally `foreach`). The portion inside the parentheses contains two parts, separated by a colon. The first is a data type and variable that will represent each of the values in the array. The second is the array name. The colon can be read as “in.” In fact, the entire foreach loop above can be read as, “For each double value temporarily called *d* in the array `cool_constants`.” The body of the foreach loop just displays the value temporarily stored in the variable *d*.

The foreach loop is a neat way to iterate over the items in an array, where each item is temporarily stored in a variable. Note that any index (i.e., location) information is not available in a foreach loop. That is, the location of the values in the array is not provided. If the index was required, then a for loop would be needed. For example:

```
for (int i = 0; i < cool_constants.length; i++)
    System.out.println(cool_constants[i]);
```

You may have noticed the condition in the for loop, `i < cool_constants.length`. This brings up the useful fact that the length of an array can be obtained by accessing its `length` member. This should clue you in that arrays must therefore be objects and have a defining class! The `length` member turns out to be an instance variable defined in the `Array` class.

Activity 2: Selection sort followed by a binary search

In this activity, we will implement the selection sort on an array, and subsequently search the sorted array for a specified value using the binary search.

Since you should be quite familiar with both the selection sort and the binary search, we will skip their mechanics and just show their algorithms. First, recall the selection sort:

```
n ← length of the list
for i ← 0..n-2
    minPosition ← i
    for j ← i+1..n-1
        if item at j < item at minPosition
            then
                minPosition ← j
```

```

        end
    next
    swap items at i and minPosition
next

```

The selection sort maintains sorted and unsorted portions of the list. It repeatedly places the smallest value in the unsorted portion of the list at the end of the sorted portion (or the beginning of the unsorted portion) of the list. The algorithm stops when the unsorted portion of the list is empty.

Second, recall the binary search:

```

num ← number to search for
first ← 0
last ← number of items in the list - 1
while first ≤ last
    mid ← floor((first + last) / 2)
    if num = item at mid of the list
    then
        return mid
    else if num > item at mid of the list
    then
        first ← mid + 1
    else
        last ← mid - 1
    end
return -1

```

The binary search selects the middle value in the current portion of the list (initially, the entire list) and compares it to the specified value. If the values match, then the (middle) index is returned. If not, and the specified value is greater than the value in the middle, then the beginning of the list is shifted to the element to the right of the middle of the list. The rest of the list is effectively discarded. Otherwise, the specified value is less than the value in the middle, and the end of the list is shifted to the element to the left of the middle of the list. Note that the algorithm has been slightly modified to return the index of the specified value (if it is found in the list) or -1 (if it isn't found in the list).

Let's write a program that generates an array of integers with random values from 10 to 99 (i.e., two digit numbers). Then, we will display the unsorted array, sort it with the selection sort, display the array again, and prompt the user for a value to search for. Finally, we will implement the binary search and return the index of the value specified (or -1 if the value is not found in the array).

First, we can layout a shell of the program. To generate random numbers, we can use the Random class (`java.util.Random`), and to accept user input from `stdin`, we can use the Scanner class (`java.util.Scanner`), as was discussed earlier. We will also apply good design principles and reduce code duplication by encapsulating common behaviors in functions. Here's a first try:

```

import java.util.Random;
import java.util.Scanner;

public class SortSearch
{

```

```

public static void main(String[] args)
{
    int[] values = new int[15];
    Random r = new Random();
    Scanner s = new Scanner(System.in);
    int num;    // the value to search for
    int index;  // the index of the specified value
}

public static void printArray(int[] list)
{
}

public static void sortArray(int[] list)
{
}

public static int searchArray(int[] list, int val)
{
}
}

```

So far, most of the functions don't contain any source code. In the main function, we declare all of the variables that we will need: the array of 15 integers, a random number generator, a scanner to obtain user input, a number to represent some value to search for in the array, and a corresponding index (if the specified value is found).

At this point, we can add statements that randomly populate the array to the main function. While we're at it, let's display the array. Here's is the updated main function (new portions have been highlighted):

```

public class SortSearch
{
    static final int MIN = 10;
    static final int MAX = 99;
    public static void main(String[] args)
    {
        int[] values = new int[15];
        Random r = new Random();
        Scanner s = new Scanner(System.in);
        int num;    // the value to search for
        int index;  // the index of the specified value

        for (int i = 0; i < values.length; i++)
            values[i] = r.nextInt(MAX - MIN + 1) + MIN;

        System.out.print("Not sorted: ");
        printArray(values);
    }
}

```

Let's explain the new statements. First, a for loop iterates over the elements of the array and assigns to each index a random integer within some range (defined by the new constants MIN and MAX that are declared and initialized at the class level).

The `nextInt` method in the `Random` class generates a random integer from 0 to one less than a specified parameter. For example, `nextInt(5)` generates a random integer from 0 to 4. Similarly, `nextInt(MAX - MIN + 1)` generates a random integer from 0 to MAX - MIN inclusive. Since MAX is 99 and MIN is 10, this is the same as `nextInt(90)`. Therefore, random integers from 0 to 89 are generated. We then add MIN to this random integer. Since MIN is 10, then the smallest integer generated is $0 + 10 = 10$, and the largest integer generated is $89 + 10 = 99$. This correctly generates random integers within the specified range of 10 to 99!

Next, we display the unsorted array. Of course, we must now implement the `printArray` function:

```
public static void printArray(int[] list)
{
    for (int i = 0; i < list.length; i++)
        System.out.print(list[i] + " ");
    System.out.println();
}
```

All this new function does is iterate over the array, and displays each element followed by a space. Lastly, it adds a newline. To be clear, the actual parameter passed in to this function in the main method is the array, *values*. It is mapped to the formal parameter specified in the function, *list*. Both are arrays of integers.

At this point, we want to sort the array, display it again (now sorted), prompt the user for a value to search the array for, search the array for this specified value, and return the index of the specified value (or -1 if it is not found in the array). We can modify the main function as follows:

```
public static void main(String[] args)
{
    int[] values = new int[15];
    Random r = new Random();
    Scanner s = new Scanner(System.in);
    int num;    // the value to search for
    int index;  // the index of the specified value

    for (int i = 0; i < values.length; i++)
        values[i] = r.nextInt((MAX - MIN) + 1) + MIN;

    System.out.print("Not sorted: ");
    printArray(values);
    sortArray(values);
    System.out.print("Sorted: ");
    printArray(values);
}
```



```

        System.out.println("Value to search for?");
        num = s.nextInt();
        index = searchArray(values, num);
        System.out.print("Value ");
        if (index == -1)
            System.out.println("not found.");
        else
            System.out.println("found at index " + index +
                               "!");
    }

```

You have seen the `nextLine` function in the `Scanner` class. We are using the `nextInt` function in this program. It will convert user input to an integer. In this case, it will be interpreted as the value to search the array for. The index is then assigned by calling the `searchArray` function, which returns an index or -1. A final string that informs the user if the specified value was found is output to the console.

Next, we will work on the `sortArray` function:

```

public static void sortArray(int[] list)
{
    for (int i = 0; i < list.length - 1; i++)
    {
        int minPos = i;

        for (int j = i + 1; j < list.length; j++)
        {
            if (list[j] < list[minPos])
                minPos = j;
        }

        int temp = list[i];
        list[i] = list[minPos];
        list[minPos] = temp;
    }
}

```

This function is an almost direct implementation of the pseudocode shown above for the selection sort. The last three statements swap the values in the array at indexes *i* and *minPos*. The strategy is to declare a temporary variable and give it the value of one of the values in the array (say at index *i*). This position in the array can now be replaced with the value at index *minPos*. Finally, the element at index *minPos* can then be replaced with the value in the temporary variable.

Lastly, we can work on the `searchArray` function that implements the binary search. Again, it is almost a direct implementation of the pseudocode shown above:

```

public static int searchArray(int[] list, int val)
{
    int first = 0;

```

```

        int last = list.length - 1;

        while (first <= last)
        {
            int mid = (first + last) / 2;

            if (val == list[mid])
                return mid;
            else if (val > list[mid])
                first = mid + 1;
            else
                last = mid - 1;
        }

        return -1;
    }
}

```

The function takes two parameters: the array to search through, and the value to search the array for. The statement, `int mid = (first + last) / 2`, works fine because it results in integer division (since *first*, *last*, and 2 are integers). If the value is found at index *mid*, then *mid* is returned. This immediately terminates the function and returns control to the main function. If the value is never found, then *first* and *last* will eventually cross, which will terminate the while loop. The value returned is then -1.

For completeness, the entire source code is shown below:

```

import java.util.Random;
import java.util.Scanner;

public class SortSearch
{
    static final int MIN = 10;
    static final int MAX = 99;

    public static void main(String[] args)
    {
        int[] values = new int[15];
        Random r = new Random();
        Scanner s = new Scanner(System.in);
        int num;    // the value to search for
        int index;  // the index of the specified value

        for (int i = 0; i < values.length; i++)
            values[i] = r.nextInt(MAX - MIN + 1) + MIN;

        System.out.print("Not sorted: ");
        printArray(values);
        sortArray(values);
    }
}

```

```

        System.out.print("Sorted: ");
        printArray(values);
        System.out.println("Value to search for?");
        num = s.nextInt();
        index = searchArray(values, num);
        System.out.print("Value ");
        if (index == -1)
            System.out.println("not found.");
        else
            System.out.println("found at index " + index +
                               "!");
    }

    public static void printArray(int[] list)
    {
        for (int i = 0; i < list.length; i++)
            System.out.print(list[i] + " ");
        System.out.println();
    }

    public static void sortArray(int[] list)
    {
        for (int i = 0; i < list.length - 1; i++)
        {
            int minPos = i;

            for (int j = i + 1; j < list.length; j++)
            {
                if (list[j] < list[minPos])
                    minPos = j;
            }

            int temp = list[i];
            list[i] = list[minPos];
            list[minPos] = temp;
        }
    }

    public static int searchArray(int[] list, int val)
    {
        int first = 0;
        int last = list.length - 1;

        while (first <= last)
        {
            int mid = (first + last) / 2;

```

```

        if (val == list[mid])
            return mid;
        else if (val > list[mid])
            first = mid + 1;
        else
            last = mid - 1;
    }

    return -1;
}

```

Of course, comments should be appropriately placed throughout the source code. They have been eliminated in this document for brevity and space considerations.

The last part of this lesson will cover the basics of implementing classes in Java. Subsequently, an example will be used to demonstrate how inheritance can be implemented.

Laying out classes

Arguably, Java is more powerful than Python when it comes to its support of the object-oriented paradigm. In fact, it was designed as a pure object-oriented language for the purpose of solving problems and designing applications using this paradigm. Python was designed more as the multi-tool for programmers so that they can get things done quickly and efficiently. Both programming languages have their strengths and their most typical intended uses.

The object-oriented principles that were covered in previous lessons are all at play in Java. For example, a class has state and behavior that collectively form its members. These members include class variables, instance variables, and methods. Classes have constructors, accessors and mutators, and can have other support methods. To practice laying out classes in Java, let's revisit the Fraction class.

Rather than listing the Python code for the Fraction class, let's itemize the requirements for this class and try to redesign it in Java. A fraction is made up of a numerator and a denominator. Instantiating one should be possible with no parameters (i.e., a default fraction of 0/1) or with specified parameters through a constructor. The denominator should never be allowed to be 0. Appropriate accessors and mutators should be provided for the numerator and denominator. Fractions should always be in their reduced form. Later, we will also need to be able to add, subtract, multiply, and divide fractions.

Let's begin with a template for the Fraction class:

```

class Fraction
{
}

```

Not too bad. The class was not specified to be public (we'll discuss why later). In Java, we place instance variables at the top of the class, outside of any methods. Let's add the fraction's instance variables now:

```

class Fraction
{
    private int numerator;
    private int denominator;
}

```

Note how the instance variables were declared to be private. This is because we do not want to allow their modification outside of the class. Instead, we will provide an accessor and mutator for each to provide read and write access.

Next, let's add the constructor. In Java, a constructor has the same name as the class, is always public (so that we can instantiate instances outside of the class), and has no return type:

```

class Fraction
{
    private int numerator;
    private int denominator;

    public Fraction()
    {
        numerator = 0;
        denominator = 1;
    }
}

```

At this point, it would be nice to test our Fraction class. The goal will be to create a program (let's call it FractionFun) that will instantiate several fractions, play around with them, and display their values throughout the program. Since in Java everything must be in a class, let's add the FractionFun class now. You can place this anywhere in the source code (i.e., either above or beneath the existing Fraction class):

```

class FractionFun
{
    public static void main(String[] args)
    {
        Fraction f = new Fraction();

        System.out.println(f);
    }
}

```

Since this class will represent our program, we must add the main method to the class. Note that we will simply be declaring and initializing a new fraction (with default values) and subsequently displaying it to the console. Here's the output after compilation and execution:

```

$ java FractionFun
Fraction@7e9bed

```

Note that, after compilation, two .class files are generated: Fraction.class and FractionFun.class. The first contains the bytecodes for the Fraction class; the second contains the bytecodes for the FractionFun

class. The latter is the one that contains the main function; therefore, this is the one that must be executed.

Note the output. It apparently makes no sense. It's actually displaying the fraction object's memory address (i.e., its location in memory). This is similar behavior to Python. Since we have not specified how to display a fraction, Java simply displays its location in memory. Just as in Python, we can override a particular method (called the `toString` method) that can return the string representation of a fraction. Let's add this function to the `Fraction` class now:

```
class Fraction
{
    private int numerator;
    private int denominator;

    public Fraction()
    {
        numerator = 0;
        denominator = 1;
    }

    public String toString()
    {
        return numerator + "/" + denominator;
    }
}
```

Recompile and execute the program again. Here's the output after the change:

```
$ java FractionFun
0/1
```

Suppose that we wanted to display the numerator of a fraction from outside of the `Fraction` class. A first try may be as follows:

```
class FractionFun
{
    public static void main(String[] args)
    {
        Fraction f = new Fraction();

        System.out.println(f.numerator);
    }
}
```

In the `FractionFun` class, we attempt to access the numerator (a member of the `Fraction` class) through the object reference, *f* (an instance of the `Fraction` class). Since the numerator is declared to be private in the `Fraction` class, it should not be possible to access it outside the class (in this case, in the `FractionFun` class). And this is exactly the case:

```
$ javac FractionFun.java
FractionFun.java:24: error: numerator has private access in Fraction
    System.out.println(f.numerator);
                        ^
1 error
```

We'll revert the FractionFun class to the previous version for now. We'll also add accessors for both of the instance variables in the Fraction class. It is convention to place accessors and mutators beneath the constructor, but above other functions:

```
class Fraction
{
    private int numerator;
    private int denominator;

    public Fraction()
    {
        numerator = 0;
        denominator = 1;
    }

    public int getNumerator()
    {
        return numerator;
    }

    public int getDenominator()
    {
        return denominator;
    }

    public String toString()
    {
        return numerator + "/" + denominator;
    }
}
```

Accessors in Java look similar to an early method of implementation that we used in Python. Accessors must be public and return the same type as the type of the instance variable it represents. We typically name the accessor using “get” followed by the name of the instance variable, initially capitalized. For the instance variable *numerator*, we name the accessor *getNumerator*. Typically, the only statement in an accessor is one that returns the value of the instance variable.

Next, let's work on mutators for the instance variables:

```
class Fraction
{
    private int numerator;
    private int denominator;
```

```

    public Fraction()
    {
        numerator = 0;
        denominator = 1;
    }

    public int getNumerator()
    {
        return numerator;
    }

    public void setNumerator(int val)
    {
        numerator = val;
    }

    public int getDenominator()
    {
        return denominator;
    }

    public void setDenominator(int val)
    {
        if (val != 0)
            denominator = val;
    }

    public String toString()
    {
        return numerator + "/" + denominator;
    }
}

```

Note how the mutators were placed beneath their respective accessors. Mutators are always public, always return nothing (since their purpose is to change a value), are usually named with “set” followed by the name of the instance variable (initially capitalized), and must have a parameter of the same type as the instance variable. This parameter will be assigned to the instance variable in the mutator.

Since the numerator can be any valid integer, we simply assign the specified value to the numerator. The denominator is a bit different. Since we cannot allow its value to be 0, we first check that it isn't 0. If this is true, we make the assignment; otherwise, we ignore the parameter.

Let's modify the FractionFun class to test what we've done so far. Here's its modified main function:


```

public static void main(String[] args)
{
    Fraction f = new Fraction();

    System.out.println(f.getDenominator());
    System.out.println(f);
    f.setNumerator(5);
    f.setDenominator(0);
    System.out.println(f);
}

```

Note that the initial value of the fraction represented by the variable *f* is 0/1. First, we display the fraction's denominator. Then, we display the entire fraction, attempt to change its numerator to 5, attempt to set its denominator to 0, and finally display the entire fraction again.

Here's the output:

```

$ java FractionFun
1
0/1
5/1

```

The output is as we expect: the Fraction class permitted the change of the numerator to 5, but not our attempt to change the denominator to 0!

Next, let's work on a way to permit the instantiation of a fraction with specified values for the numerator and denominator. There are several ways of doing this. One involves adding a second constructor with parameters that will be mapped to the instance variables:

```

class Fraction
{
    private int numerator;
    private int denominator;
    public Fraction()
    {
        numerator = 0;
        denominator = 1;
    }

    public Fraction(int n, int d)
    {
        numerator = n;
        if (d != 0)
            denominator = d;
        else
            denominator = 1;
    }
}

```

```

    public int getNumerator()
    {
        return numerator;
    }

    public void setNumerator(int val)
    {
        numerator = val;
    }

    public int getDenominator()
    {
        return denominator;
    }

    public void setDenominator(int val)
    {
        if (val != 0)
            denominator = val;
    }

    public String toString()
    {
        return numerator + "/" + denominator;
    }
}

```

How does Java know which constructor to use? Well, if we initialize a fraction without providing any parameters, it uses the default one that has no parameters; otherwise, it uses this new version. Java permits functions of the same name, so long as their signatures are different. This usually means that their parameters must differ so that the compiler knows which function to use. Note that we ensure that, if the specified denominator is 0, we ignore it and force the fraction's denominator to be 1.

To show that the new constructor works, the main function of the FractionFun class has been replaced as follows:

```

public static void main(String[] args)
{
    Fraction f = new Fraction(1, 2);

    System.out.println(f);
    f = new Fraction(5, 0);
    System.out.println(f);
}

```

And here's the output, as expected:

```

$ java FractionFun
1/2
5/1

```

Note how we can reinitialize the fraction, f , with the statement, `f = new Fraction(5, 0)`.

Next, we should probably work on a function that reduces a fraction. You have already done this in the Python assignment; therefore, we will simply take the Python source code and convert it to Java. If necessary, please refer to the first lesson on the object-oriented paradigm that discusses the Fraction class. In addition, we will add appropriate calls to this new function in the constructor (with parameters) and in the mutators:

```
class Fraction
{
    private int numerator;
    private int denominator;

    public Fraction()
    {
        numerator = 0;
        denominator = 1;
    }

    public Fraction(int n, int d)
    {
        numerator = n;
        if (d != 0)
            denominator = d;
        else
            denominator = 1;
        reduce();
    }

    public int getNumerator()
    {
        return numerator;
    }

    public void setNumerator(int val)
    {
        numerator = val;
        reduce();
    }

    public int getDenominator()
    {
        return denominator;
    }

    public void setDenominator(int val)
    {
        if (val != 0)
```

```

        {
            denominator = val;
            reduce();
        }
    }

    private void reduce()
    {
        int gcd = 1;
        int min = Math.min(Math.abs(numerator),
                           Math.abs(denominator));

        for (int i = 2; i <= min; i++)
        {
            if (numerator % i == 0 && denominator % i == 0)
                gcd = i;
        }

        numerator /= gcd;
        denominator /= gcd;

        if (numerator == 0)
            denominator = 1;
    }

    public String toString()
    {
        return numerator + "/" + denominator;
    }
}

```

Note how the reduce function is private. There is no need to make it publicly accessible outside of the class, since it will only be used internally as fractions are instantiated or instance variables are modified. It is good programming practice to publicly expose only required functions. The rest should be private. Also note that the reduce function makes use of Java's Math library. Specifically, it uses its **min** and **abs** functions to correctly determine the minimum between the function's numerator and denominator.

We will also replace the main function of the FractionFun class as follows to test the new reduce function:

```

public static void main(String[] args)
{
    Fraction f = new Fraction(2, 4);

    System.out.println(f);
    f = new Fraction(45, 198);
    System.out.println(f);
}

```

And here's the output, as expected:

```
$ java FractionFun
1/2
5/22
```

Next, let's add a function that returns the floating point representation of a fraction, and modify the string representation appropriately. Add this new function in the Fraction class above the reduce function:

```
private double getReal()
{
    return (double)numerator / denominator;
}
```

This new function is also private, since it has no use outside of the Fraction class. We'll also change the string representation of a fraction as follows:

```
public String toString()
{
    return numerator + "/" + denominator + " (" +
        getReal() + ")";
}
```

And here's the output with the above changes:

```
$ java FractionFun
1/2 (0.5)
5/22 (0.22727272727272727)
```

Lastly, let's add support for arithmetic (addition, subtraction, multiplication, and division) with fractions. Recall that, in Python, we were able to **overload** the various arithmetic operators using special magic functions. We could then add fractions using the operator symbols (e.g., +, -, *, and /). Unfortunately, the same is not possible in Java. Therefore, we will resort to functions with appropriate names to accomplish the same thing. Here are the four arithmetic functions. They should be placed above the getReal function and beneath the accessors and mutators. For a discussion on the logic of these functions, please refer to the previous lesson that discusses the Fraction class:

```
public Fraction add(Fraction f)
{
    Fraction sum = new Fraction();

    sum.numerator = (numerator * f.denominator) +
        (denominator * f.numerator);
    sum.denominator = denominator * f.denominator;
    sum.reduce();

    return sum;
}
```

```

public Fraction sub(Fraction f)
{
    Fraction neg_f = new Fraction(f.numerator * -1,
                                   f.denominator);
    Fraction diff = this.add(neg_f);

    return diff;
}

public Fraction mul(Fraction f)
{
    Fraction prod = new Fraction();

    prod.numerator = numerator * f.numerator;
    prod.denominator = denominator * f.denominator;
    prod.reduce();

    return prod;
}

public Fraction div(Fraction f)
{
    Fraction f_reciprocal = new Fraction(f.denominator,
                                           f.numerator);
    Fraction div = this.mul(f_reciprocal);

    return div;
}

```

If you're wondering why we can access the private instance variables *numerator* and *denominator* of fraction *f* passed in as a parameter to these functions, it is because we are in the Fraction class! Even though we are referring to a separate instance of the Fraction class, we can still access its private members.

You probably also noticed a new keyword, *this*, in the sub and div functions. Think of *this* in Java as the same thing as *self* in Python. It refers to the *current* instance of a fraction. Consider, for example, the following statements in the main function of the FractionFun class:

```

Fraction f1 = new Fraction(1, 2);
Fraction f2 = new Fraction(2, 3);
System.out.println(f1.sub(f2));

```

In the `System.out.println` statement, fraction *f1* invokes its sub function, passing fraction *f2* as a parameter. In the sub function, *f2* is mapped to *f* in the parameter list. The keyword *this* refers to the current instance of the fraction that invoked the method. In this case, it is fraction *f1*. Lastly, let's update the main function of the FractionFun class so that it tests the new arithmetic functions:

```

public static void main(String[] args)
{
    Fraction f1 = new Fraction();
    Fraction f2 = new Fraction(5, 8);
    Fraction f3 = new Fraction(3, 4);
    Fraction f4 = new Fraction(1, 0);

    System.out.println("f1: " + f1);
    System.out.println("f2: " + f2);
    System.out.println("f3: " + f3);
    System.out.println("f4: " + f4);

    f3.setNumerator(5);
    f3.setDenominator(8);
    f1 = f2.add(f3);
    f4.setDenominator(88);
    f2 = f1.sub(f1);
    f3 = f1.mul(f1);
    f4 = f4.div(f3);

    System.out.println();
    System.out.println("f1: " + f1);
    System.out.println("f2: " + f2);
    System.out.println("f3: " + f3);
    System.out.println("f4: " + f4);
}

```

And here's the output:

```

$ java FractionFun
f1: 0/1 (0.0)
f2: 5/8 (0.625)
f3: 3/4 (0.75)
f4: 1/1 (1.0)

f1: 5/4 (1.25)
f2: 0/1 (0.0)
f3: 25/16 (1.5625)
f4: 2/275 (0.0072727272727273)

```

For completeness, here is the entire updated Fraction and FractionFun classes:

```

class Fraction
{
    private int numerator;
    private int denominator;

    public Fraction()
    {

```

```

        numerator = 0;
        denominator = 1;
    }

    public Fraction(int n, int d)
    {
        numerator = n;
        if (d != 0)
            denominator = d;
        else
            denominator = 1;
        reduce();
    }

    public int getNumerator()
    {
        return numerator;
    }

    public void setNumerator(int val)
    {
        numerator = val;
        reduce();
    }

    public int getDenominator()
    {
        return denominator;
    }

    public void setDenominator(int val)
    {
        if (val != 0)
        {
            denominator = val;
            reduce();
        }
    }

    public Fraction add(Fraction f)
    {
        Fraction sum = new Fraction();

        sum.numerator = (numerator * f.denominator) +
            (denominator * f.numerator);
        sum.denominator = denominator * f.denominator;
        sum.reduce();
    }

```



```

        return sum;
    }

    public Fraction sub(Fraction f)
    {
        Fraction neg_f = new Fraction(f.numerator * -1,
                                       f.denominator);
        Fraction diff = this.add(neg_f);

        return diff;
    }

    public Fraction mul(Fraction f)
    {
        Fraction prod = new Fraction();

        prod.numerator = numerator * f.numerator;
        prod.denominator = denominator * f.denominator;
        prod.reduce();

        return prod;
    }

    public Fraction div(Fraction f)
    {
        Fraction f_reciprocal = new Fraction(f.denominator,
                                             f.numerator);
        Fraction div = this.mul(f_reciprocal);

        return div;
    }

    private double getReal()
    {
        return (double)numerator / denominator;
    }

    private void reduce()
    {
        int gcd = 1;
        int min = Math.min(Math.abs(numerator),
                           Math.abs(denominator));

        for (int i=2; i<=min; i++)
        {
            if (numerator % i == 0 && denominator % i == 0)
                gcd = i;
        }
    }

```

```

    }

    numerator /= gcd;
    denominator /= gcd;

    if (numerator == 0)
        denominator = 1;
}

public String toString()
{
    return numerator + "/" + denominator + " (" +
        getReal() + ")";
}
}

class FractionFun
{
    public static void main(String[] args)
    {
        Fraction f1 = new Fraction();
        Fraction f2 = new Fraction(5, 8);
        Fraction f3 = new Fraction(3, 4);
        Fraction f4 = new Fraction(1, 0);

        System.out.println("f1: " + f1);
        System.out.println("f2: " + f2);
        System.out.println("f3: " + f3);
        System.out.println("f4: " + f4);

        f3.setNumerator(5);
        f3.setDenominator(8);
        f1 = f2.add(f3);
        f4.setDenominator(88);
        f2 = f1.sub(f1);
        f3 = f1.mul(f1);
        f4 = f4.div(f3);

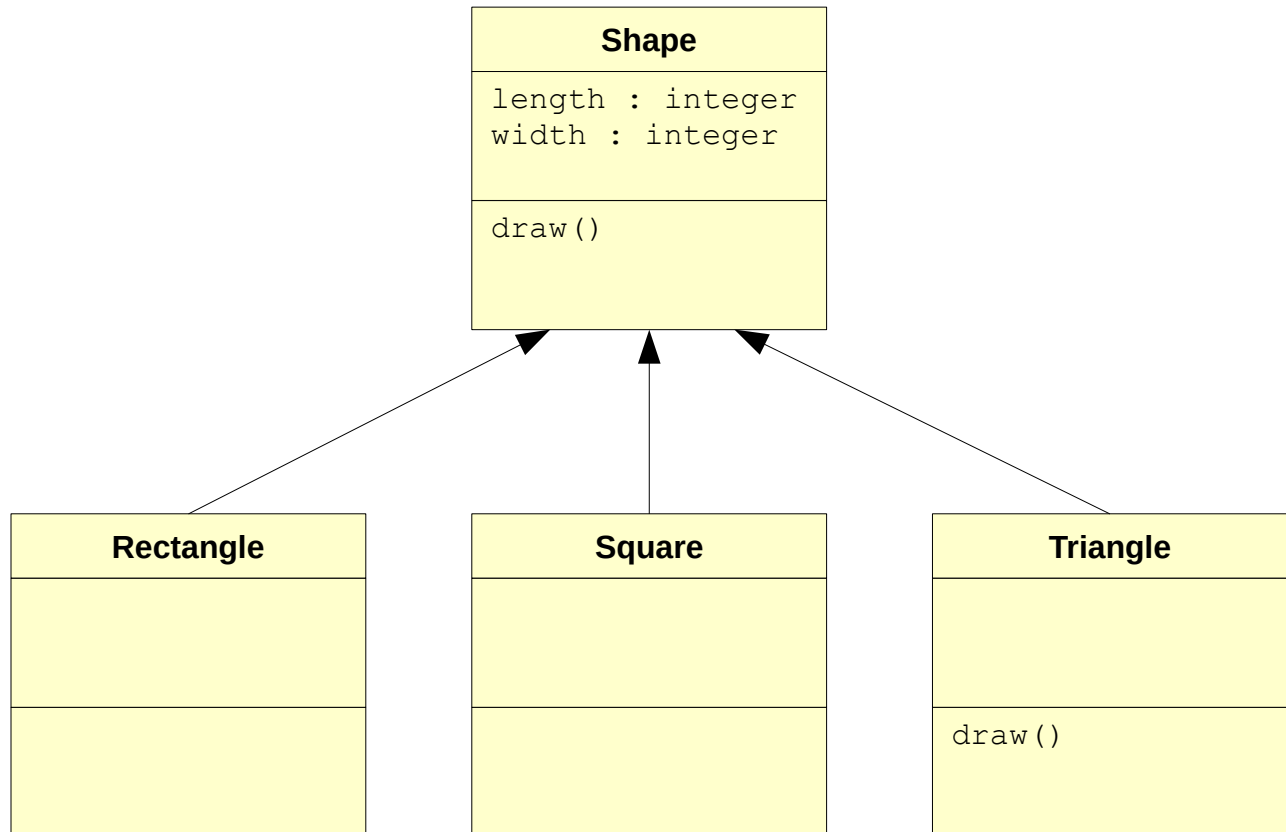
        System.out.println();
        System.out.println("f1: " + f1);
        System.out.println("f2: " + f2);
        System.out.println("f3: " + f3);
        System.out.println("f4: " + f4);
    }
}

```

Inheritance, abstract classes and methods, and more

Since Java is an object-oriented programming language, all of the concepts that you have learned about the object-oriented paradigm are relevant. The difference between Java and Python (or any other object-oriented programming language) is just in how it syntactically implements the concepts. The basic idea of superclasses and subclasses exists. In fact, their implementation in Java is pretty similar to Python.

To illustrate how Java implements inheritance, let's revisit the example of the shapes project introduced in an earlier lesson on the object-oriented paradigm. In this example, we defined four classes: a Shape class that encapsulated the members that all shapes shared, a Rectangle class that specifically described a rectangle, a Square class, and a Triangle class. We restricted triangles to those that are right-angled isosceles triangles.



Since rectangles and squares are similar, we specified the draw function in the Shape superclass. Triangles, however, were a bit different; therefore, we overloaded the draw function (defined in the Shape superclass) in the Triangle subclass. Feel free to refer to the previous lesson if necessary.

Let's begin by reimplementing the Shape class in Java:

```
abstract class Shape
{
    protected int length;
    protected int width;
```

```

    public Shape(int l, int w)
    {
        length = l;
        width = w;
    }

    public void draw()
    {
        for (int i = 0; i < width; i++)
        {
            for (int j = 0; j < length; j++)
                System.out.print("* ");
            System.out.println();
        }
    }
}

```

Most of the Shape class should be familiar to you. There are, however, a few new surprises. First, the keyword `abstract` in the class header: `abstract class Shape`. In Java, this keyword marks a class as abstract. Recall that an abstract class cannot be instantiated. This makes sense, because our intention is to instantiate rectangles, squares, and triangles (and not shapes in general). The Shape class is only used to encapsulate members common to all of the shapes.

Second, the keyword `protected` is used to specify a different level of accessibility for the instance variables. You are already familiar with `public` and `private`. The `protected` modifier stretches the accessibility of a member to all subclasses of the current class. To summarize, public members can be accessed anywhere, private members can only be accessed in the class that defines them, and protected members can be accessed in the class that defines them and all subclasses of the class that defines them. Therefore, protected members in the Shape class can be accessed in the Shape class (since it defines them), and the Rectangle, Square, and Triangle classes (since they are subclasses of the Shape class).

The Shape class has a constructor that takes a length and a width of the shape to be instantiated. It also has a draw method that draws the shape using asterisks as specified in the earlier lesson.

Now, let's work on the individual shape classes. First, the Rectangle class:

```

class Rectangle extends Shape
{
    public Rectangle(int l, int w)
    {
        super(l, w);
    }
}

```

The method that Java uses to create a subclass involves the use of the keyword `extends`. In fact, this makes sense because a subclass does indeed *extend* its superclass. The class header, `class Rectangle extends Shape`, marks the Rectangle class as a subclass of the Shape class.

Therefore, the Rectangle class will inherit all members defined in the Shape class. It will also be able to access all public and protected members in the Shape class.

The Rectangle class only has a constructor that accepts a rectangle's length and width. The single statement in the constructor calls the superclass' constructor, passing in those same parameters. The keyword `super` in Java means to access the superclass. The statement, `super()`, calls the constructor in the superclass. In fact, this is actually done by default, and we don't even have to add the statement if the constructor has no parameters. Since the Shape class' constructor takes two parameters, we must formally call the superclass' constructor, passing in the two parameters.

Next, let's work on the Square class. In fact, it looks very similar to the rectangle class, except that a square has the same length and width:

```
class Square extends Shape
{
    public Square(int s)
    {
        super(s, s);
    }
}
```

The variable, `s`, passed as parameter to the constructor in the Square class denotes its size. It is passed twice in the call to the superclass' constructor.

Lastly, the triangle class:

```
class Triangle extends Shape
{
    public Triangle(int s)
    {
        super(s, s);
    }
    public void draw()
    {
        for (int i = 0; i < width; i++)
        {
            for (int j = 0; j < width - i; j++)
                System.out.print("* ");
            System.out.println();
        }
    }
}
```

The Triangle class is slightly different than the other two shape classes. Since it is drawn differently than the others, we override the draw function with a new one that describes specifically how to draw a triangle. The rest of the class is similar to the other shape classes.

Finally, we can create a ShapeTest class that will serve as our application, and will instantiate several of the shapes and draw them to the console:

```

public class ShapeTest
{
    public static void main(String[] args)
    {
        Rectangle r = new Rectangle(12, 4);
        Square s = new Square(6);
        Triangle t = new Triangle(7);

        r.draw();
        System.out.println();
        s.draw();
        System.out.println();
        t.draw();
    }
}

```

Note that all of the classes above can be placed in the same Java source file. In Java, however, if a class is qualified with the public modifier, then it must be saved in a file that is of the same name as the class. You can see how this could be a problem if, for example, all of the classes above had been qualified with the public modifier. Of course, they were not; however, the ShapeTest class was. Therefore, the file must be saved as ShapeTest.java. In addition, the ShapeTest class must contain the main function since it will be used to execute the program. The ShapeTest class simply instantiates each shape, specifying various shape sizes. The shapes are then drawn.

Here is the output:

```

$ java ShapeTest
* * * * *
* * * * *
* * * * *
* * * * *

* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *

* * * * *
* * * * *
* * * * *
* * * *
* * *
* *
*

```

For completeness, here is the entire source code to the shapes project:

```
abstract class Shape
{
    protected int length;
    protected int width;

    public Shape(int l, int w)
    {
        length = l;
        width = w;
    }

    public void draw()
    {
        for (int i = 0; i < width; i++)
        {
            for (int j = 0; j < length; j++)
                System.out.print("* ");
            System.out.println();
        }
    }
}

class Rectangle extends Shape
{
    public Rectangle(int l, int w)
    {
        super(l, w);
    }
}

class Square extends Shape
{
    public Square(int s)
    {
        super(s, s);
    }
}

class Triangle extends Shape
{
    public Triangle(int s)
    {
        super(s, s);
    }

    public void draw()
```

```

    {
        for (int i = 0; i < width; i++)
        {
            for (int j = 0; j < width - i; j++)
                System.out.print("* ");
            System.out.println();
        }
    }
}

public class ShapeTest
{
    public static void main(String[] args)
    {
        Rectangle r = new Rectangle(12, 4);
        Square s = new Square(6);
        Triangle t = new Triangle(7);

        r.draw();
        System.out.println();
        s.draw();
        System.out.println();
        t.draw();
    }
}

```

Just to show that abstract classes cannot be instantiated, let's add the following statement in the main function of the ShapeTest class:

```
Shape poly = new Shape(5, 15);
```

Here's the output (as expected) when we try to compile the program:

```

$ javac ShapeTest.java
ShapeTest.java:71: error: Shape is abstract; cannot be instantiated
    Shape poly = new Shape(5, 15);
                   ^
1 error

```

The compiler enforces the abstract class and prevents its instantiation.

In the previous lesson on the object-oriented paradigm, we also discussed abstract methods. These are methods that are defined in an abstract class, and that *must* be implemented in all subclasses of the class that defines them. Although we don't have any abstract methods in the shapes project above, let's add one to the Shape class to see what happens if the individual shape subclasses don't implement them. We'll add a fictitious abstract method, `foo`, to the Shape class and recompile the program. Since the point of an abstract class is to implement it in subclasses, then there is no body to the method specified in the superclass. Instead, we only provide its signature and terminate it with a semicolon:


```

abstract class Shape
{
    protected int length;
    protected int width;

    public Shape(int l, int w)
    {
        length = l;
        width = w;
    }

    ...

    abstract void foo();
}

```

Here's the output when compiling the program:

```

$ javac ShapeTest.java
ShapeTest.java:25: error: Rectangle is not abstract and does not
override abstract method foo() in Shape
class Rectangle extends Shape
^
ShapeTest.java:33: error: Square is not abstract and does not
override abstract method foo() in Shape
class Square extends Shape
^
ShapeTest.java:41: error: Triangle is not abstract and does not
override abstract method foo() in Shape
class Triangle extends Shape
^
3 errors

```

The errors produced are very clear and expected. The shape subclasses must override the abstract method `foo`. To fix this, we would simply need to define the `foo` method in each of the subclasses.

Finally, we discussed the idea of multiple inheritance, where a class can inherit traits from more than one superclass. This was useful, for example, in a grocery store's point-of-sale system where a banana could be both a fruit (with a point of origin, date of harvest, etc) and a grocery store item (with a price, location, etc). In Java, multiple inheritance is not directly supported. It can be *faked*, however, through the use of interfaces. In Java, an **interface** is like a contract. You will learn about them in future classes, and discussing them here is beyond the scope of this lesson.